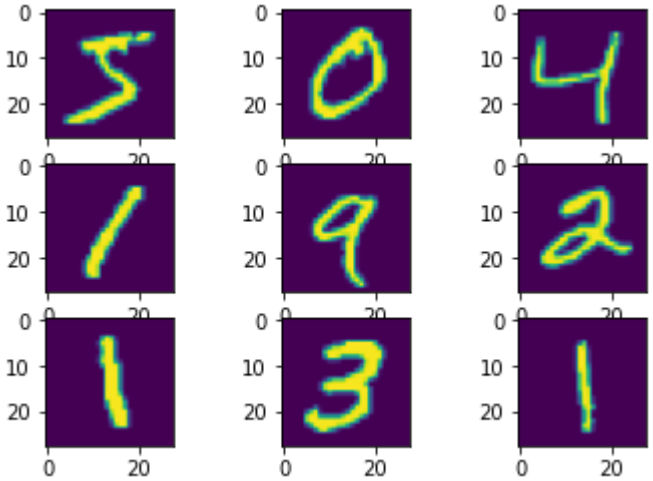
In [ ]:
```
import keras
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D,Dense, Flatten
from keras.utils import to_categorical
from keras.datasets import mnist
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
```

In [2]:
```
(xtrain,ytrain),(xtest,ytest)=mnist.load_data()
```

In [3]:
```
xtrain=xtrain.reshape((xtrain.shape[0],28,28,1))
xtest=xtest.reshape((xtest.shape[0],28,28,1))
```

In [4]:
```
train_norm = xtrain.astype('float32')
test_norm = xtest.astype('float32')
xtrain = train_norm/255.0
xtest = test_norm/255.0
```

In [5]:
```
for i in range(9):
  plt.subplot(330+1+i)
  plt.imshow(xtrain[i])
```



In [8]:
```
model = Sequential()
model.add(Conv2D(32,(3,3),activation='relu',input_shape=(28,28,1)))
model.add(MaxPooling2D(2,2))
model.add(Flatten())
model.add(Dense(100,activation='relu'))
model.add(Dense(10,activation='softmax'))
```

In [9]:
```
model.compile(optimizer='sgd',loss='sparse_categorical_crossentropy',metr
```

In [10]:
```
history=model.fit(xtrain,ytrain,epochs=5,validation_data=(xtest,ytest))
```

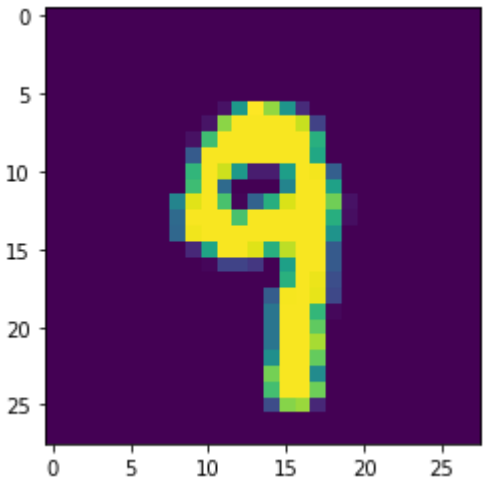
```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8744 - loss: 0.46
61 - val_accuracy: 0.9285 - val_loss: 0.2453
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9334 - loss: 0.22
32 - val_accuracy: 0.9480 - val_loss: 0.1783
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9490 - loss: 0.16
99 - val_accuracy: 0.9559 - val_loss: 0.1552
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9585 - loss: 0.13
94 - val_accuracy: 0.9625 - val_loss: 0.1223
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9635 - loss: 0.12
06 - val_accuracy: 0.9663 - val_loss: 0.1088
```

In [11]:
```
predictions =model.predict(xtest)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
```

In [12]:
```
import random
index = random.randint(1,10000)
plt.imshow(xtest[index])
```

Out[12]: `<matplotlib.image.AxesImage at 0x7d817c6210f0>`



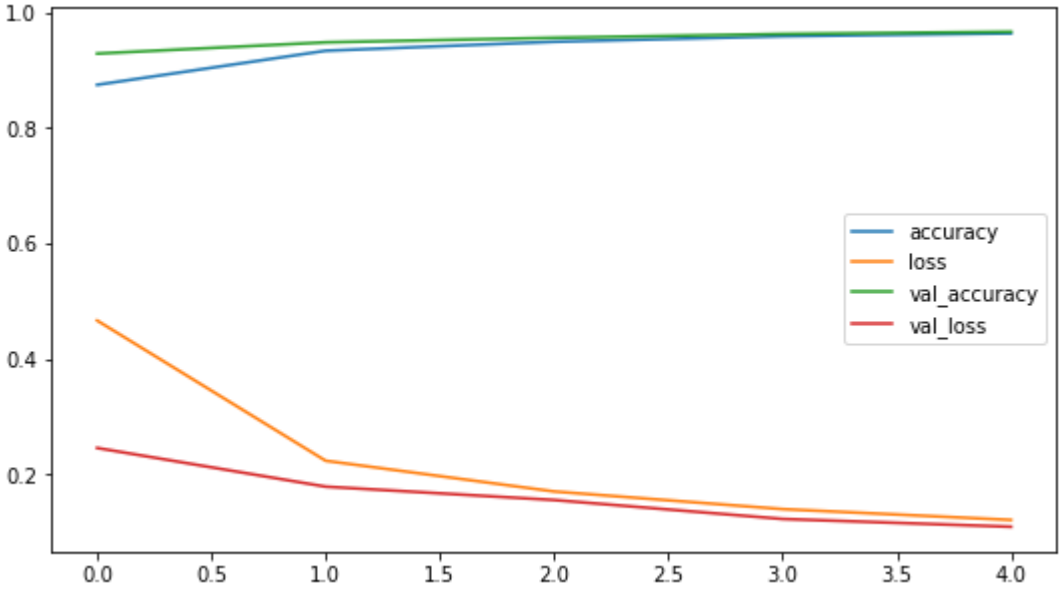
In [13]:
```
pred_num = max(predictions[index])
for i in range(len(predictions[index])):
    if(predictions[index][i] == pred_num):
        print("Predicted number is: ",i)
        break
```

```
Predicted number is:  9
```

In [14]:
```
print(model.evaluate(xtest,ytest))
```

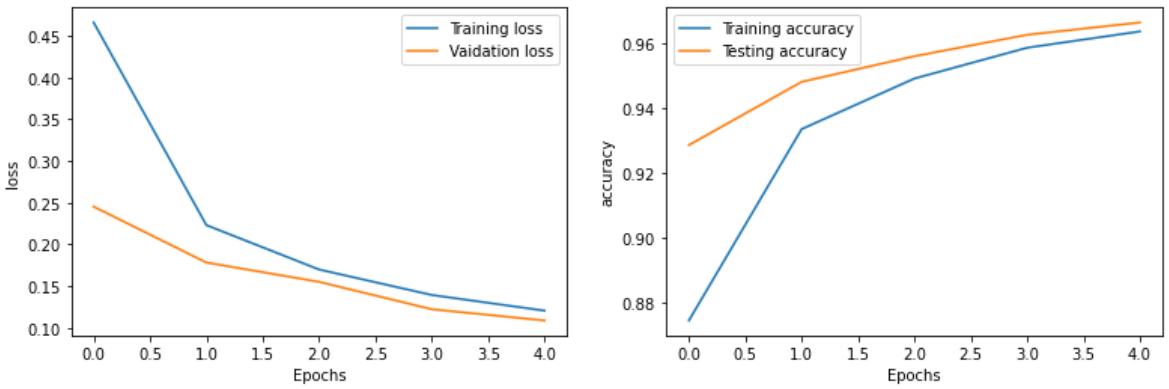
```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9663 - loss: 0.1088
[0.10882052034139633, 0.9663000106811523]
```

In [15]:
```
import pandas as pd
pd.DataFrame(history.history).plot(figsize=(9,5))
plt.show()
```



In [16]:
```
plt.figure(figsize=(13,4))
plt.subplot(1,2,1)
plt.plot(history.history['loss'],label='Training loss')
plt.plot(history.history['val_loss'],label='Vaidation loss')
plt.xlabel('Epochs')
plt.ylabel('loss')
plt.legend()
plt.subplot(1,2,2)
plt.plot(history.history['accuracy'],label='Training accuracy')
plt.plot(history.history['val_accuracy'],label='Testing accuracy')
plt.xlabel('Epochs')
plt.ylabel('accuracy')
plt.legend()
```

Out[16]: <matplotlib.legend.Legend at 0x7d817c402ec0>



In [ ]: